

Yes—you should start now, and **Python first is the right choice**.

Power BI is excellent for communicating the findings, but Python is better for:

- cleaning and restructuring the data;
- matching hourly weather to 5-minute plant readings;
- checking missing values, duplicates, and outliers;
- correlation and statistical testing;
- regression and nonlinear driver analysis;
- producing a clean table for Power BI.

You do not need to emphasize that Python was not in the job description. Present it as an analytical quality-control step:

“I used Python for data preparation and statistical analysis, then used Power BI to communicate the operational insights.”

That sounds stronger than trying to do everything inside Power BI.

Important correction: hourly, not daily weather

You should not assign one **daily temperature** to every 5-minute reading. The weather dataset is hourly, so each 5-minute operating record should receive the weather observation for the corresponding hour.

For example:

Plant timestamp	Matched weather timestamp
10:00	10:00
10:05	10:00
10:10	10:00
10:55	10:00
11:00	11:00

Your workbook contains:

- **4,032 operational observations**, corresponding to 14 days of 5-minute readings;
- **336 weather observations**, corresponding to 14 days of hourly readings.

This is internally consistent because:

- 14 days × 24 hours × 12 five-minute intervals = 4,032;
- 14 days × 24 hours = 336.

Also, based on the uploaded workbook, **Plant 2 does not appear to be binary**. Its values are continuous and range approximately from **-440 to 6,686 tons**. Plant 1 contains numeric values including many zeros. Therefore, keep your clarification question, but adjust it because Plant 2 is not simply 0/1 in the workbook I inspected.

Recommended project workflow

Use this sequence:

1. Load and separate operational and weather data.
2. Rename columns clearly.
3. Validate timestamps and data types.
4. Join weather to 5-minute plant readings.
5. Check missing values and duplicates.
6. Examine unusual and potentially invalid values.
7. Create operational features.
8. Explore distributions and relationships.
9. Run correlation analysis.
10. Build regression models.
11. Test nonlinear effects and interactions.
12. Export the final analysis table to Power BI.
13. Build the dashboard and presentation.

Part 1: Jupyter Notebook setup

Create a new notebook named:

customer_dp_driver_analysis.ipynb



Cell 1: Import libraries

Python



Run

```
from pathlib import Path
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from scipy import stats
```

```

from sklearn.model_selection import (
    TimeSeriesSplit,
    cross_validate
)
from sklearn.pipeline import Pipeline
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.compose import ColumnTransformer
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor, HistGradientBoostingRegressor
from sklearn.inspection import permutation_importance, PartialDependenceDisplay
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor
from statsmodels.stats.diagnostic import het_breuschpagan

warnings.filterwarnings("ignore")

pd.set_option("display.max_columns", 100)
pd.set_option("display.float_format", lambda x: f"{x:,.3f}")

```

Install missing packages only when required:

<> Python



▶ Run

```

# Run only if a package is missing:
# !pip install pandas openpyxl matplotlib scipy scikit-learn statsmodels

```

Cell 2: Set the file path

<> Python



▶ Run

```

DATA_PATH = Path("Data Set for Ana.xlsx")

if not DATA_PATH.exists():
    raise FileNotFoundError(
        f"File not found: {DATA_PATH.resolve()}\n"
        "Place the Excel file in the same folder as the notebook."
    )

print(f"Using file: {DATA_PATH.resolve()}")

```

Part 2: Read and restructure the workbook

The actual column headers begin on Excel row 6, so we use `header=5`.

Cell 3: Read the worksheet

Python



Run

```
raw = pd.read_excel(
    DATA_PATH,
    sheet_name="Data set",
    header=5
)

print(f"Raw shape: {raw.shape}")
display(raw.head())
```

You should see approximately:

(4032, 13)



The operations and weather data are positioned side-by-side, but they are logically two separate tables.

Cell 4: Separate operational and weather data

Python



Run

```
operations = raw.iloc[:, 1:10].copy()
weather = raw.iloc[:, 11:13].copy()

operations.columns = [
    "timestamp",
    "customer_dp_psi",
    "system_output_pressure_psi",
    "system_return_temp_f",
    "system_supply_temp_f",
    "total_system_cooling_tons",
    "total_system_flow_gpm",
    "plant_1_instantaneous_tons",
    "plant_2_instantaneous_tons",
]

weather.columns = [
    "weather_timestamp",
    "humidex_c",
]
```

```
print("Operations shape:", operations.shape)
print("Weather shape before removing blank rows:", weather.shape)

display(operations.head())
display(weather.head())
```

Cell 5: Remove blank weather rows

The weather columns contain only 336 valid rows; the remaining rows are blank because the weather readings are hourly.

Python



Run

```
weather = weather.dropna(
    subset=["weather_timestamp", "humidex_c"]
).copy()

print("Operations observations:", len(operations))
print("Weather observations:", len(weather))
```

Expected result:

```
Operations observations: 4032
Weather observations: 336
```



Part 3: Validate and clean data types

Cell 6: Convert timestamps and numeric columns

Python



Run

```
operations["timestamp"] = pd.to_datetime(
    operations["timestamp"],
    errors="coerce"
)

weather["weather_timestamp"] = pd.to_datetime(
    weather["weather_timestamp"],
    errors="coerce"
)

operation_numeric_columns = [
    col for col in operations.columns
    if col != "timestamp"
```

```

]

operations[operation_numeric_columns] = operations[
    operation_numeric_columns
].apply(pd.to_numeric, errors="coerce")

weather["humidex_c"] = pd.to_numeric(
    weather["humidex_c"],
    errors="coerce"
)

print(operations.dtypes)
print()
print(weather.dtypes)

```

Cell 7: Check timestamp coverage and intervals

<> Python



▶ Run

```

print("Operational period:")
print("Start:", operations["timestamp"].min())
print("End:  ", operations["timestamp"].max())

print("\nWeather period:")
print("Start:", weather["weather_timestamp"].min())
print("End:  ", weather["weather_timestamp"].max())

operations_interval = (
    operations["timestamp"]
    .sort_values()
    .diff()
    .dropna()
    .dt.total_seconds()
    .div(60)
)

weather_interval = (
    weather["weather_timestamp"]
    .sort_values()
    .diff()
    .dropna()
    .dt.total_seconds()
    .div(60)
)

print("\nMost common operational interval:")
print(operations_interval.value_counts().head())

```

```
print("\nMost common weather interval:")
```

You should confirm:

- operational interval is usually 5 minutes;
- weather interval is usually 60 minutes.

Part 4: Join hourly weather to 5-minute plant readings

There are two defensible methods.

Preferred method: match by hour

This method assigns each hourly reading to all 5-minute records within that same hour.

Cell 8: Create hourly matching keys

<> Python



▶ Run

```
operations["weather_hour"] = operations["timestamp"].dt.floor("h")
weather["weather_hour"] = weather["weather_timestamp"].dt.floor("h")

analysis_df = operations.merge(
    weather[["weather_hour", "humidex_c"]],
    on="weather_hour",
    how="left",
    validate="many_to_one"
)

print("Merged shape:", analysis_df.shape)
print("Missing humidex values:", analysis_df["humidex_c"].isna().sum())

display(
    analysis_df[
        ["timestamp", "weather_hour", "humidex_c"]
    ].head(15)
)
```

The `validate="many_to_one"` argument is useful. It confirms that:

- many 5-minute operational readings can match one hourly weather reading;
- each weather hour must be unique.

Verify the join

<> Python



▶ Run

```
weather_hour_counts = analysis_df.groupby("weather_hour").size()
```

```
print(weather_hour_counts.describe())
```

Most complete hours should have approximately **12 operational records**.

Alternative method: use the most recent weather observation

This approach is useful when timestamps are not perfectly aligned to the hour.

Python



Run

```
operations_sorted = operations.sort_values("timestamp")
weather_sorted = weather.sort_values("weather_timestamp")

analysis_asof = pd.merge_asof(
    operations_sorted,
    weather_sorted,
    left_on="timestamp",
    right_on="weather_timestamp",
    direction="backward",
    tolerance=pd.Timedelta("1 hour")
)

print(
    "Missing humidex after as-of join:",
    analysis_asof["humidex_c"].isna().sum()
)
```

For this case, I would initially use the **hour-floor method** because the weather observations are clearly hourly. Mention the method in your presentation:

“Hourly weather observations were matched to all operational records occurring within the same hour.”

Part 5: Data-quality assessment

Cell 9: Create a data-quality summary

Python



Run

```
def quality_summary(df):
    summary = pd.DataFrame({
        "data_type": df.dtypes.astype(str),
        "row_count": len(df),
        "non_null_count": df.notna().sum(),
        "missing_count": df.isna().sum(),
        "missing_pct": df.isna().mean() * 100,
```

```

        "unique_values": df.nunique(dropna=True),
    })

    return summary.sort_values(
        ["missing_pct", "unique_values"],
        ascending=[False, True]
    )

quality_report = quality_summary(analysis_df)

display(quality_report)

```

Cell 10: Check duplicates

<> Python



▶ Run

```

full_duplicate_count = analysis_df.duplicated().sum()
timestamp_duplicate_count = analysis_df["timestamp"].duplicated().sum()

print("Fully duplicated rows:", full_duplicate_count)
print("Duplicated operational timestamps:", timestamp_duplicate_count)

```

Do not immediately delete duplicate timestamps. A duplicated timestamp may be legitimate when multiple systems or customers are present, although this dataset appears to represent one customer.

Use this decision rule:

<> Python



▶ Run

```

if full_duplicate_count > 0:
    analysis_df = analysis_df.drop_duplicates().copy()
    print("Exact duplicate rows removed.")
else:
    print("No exact duplicate rows found.")

```

Cell 11: Check numeric ranges

<> Python



▶ Run

```

numeric_columns = analysis_df.select_dtypes(
    include="number"
).columns.tolist()

range_summary = analysis_df[numeric_columns].agg(

```

```
["count", "mean", "std", "min", "median", "max"]
).T

display(range_summary)
```

Based on the workbook, you should immediately note:

- Customer DP ranges approximately from **5.68 to 36.31 psi**.
- Mean DP is approximately **17.51 psi**, above the stated 12 psi preference.
- Plant 1 includes many zero readings.
- Plant 2 includes negative readings.
- Negative Plant 2 values should not automatically be deleted until their operational meaning is confirmed.

Cell 12: Investigate unusual plant values

</> Python



▶ Run

```
plant_checks = pd.DataFrame({
    "variable": [
        "plant_1_instantaneous_tons",
        "plant_2_instantaneous_tons"
    ],
    "minimum": [
        analysis_df["plant_1_instantaneous_tons"].min(),
        analysis_df["plant_2_instantaneous_tons"].min()
    ],
    "maximum": [
        analysis_df["plant_1_instantaneous_tons"].max(),
        analysis_df["plant_2_instantaneous_tons"].max()
    ],
    "zero_count": [
        analysis_df["plant_1_instantaneous_tons"].eq(0).sum(),
        analysis_df["plant_2_instantaneous_tons"].eq(0).sum()
    ],
    "negative_count": [
        analysis_df["plant_1_instantaneous_tons"].lt(0).sum(),
        analysis_df["plant_2_instantaneous_tons"].lt(0).sum()
    ]
})

plant_checks["zero_pct"] = (
    plant_checks["zero_count"] / len(analysis_df) * 100
)

plant_checks["negative_pct"] = (
    plant_checks["negative_count"] / len(analysis_df) * 100
)
```

```
display(plant_checks)
```

Your notebook should contain an explicit note:

</> Python



▶ Run

```
DATA_ASSUMPTIONS = """
1. Zero plant tons may indicate that the plant was off, unavailable,
   or not supplying load.
2. Negative tonnage values will be retained until their operational
   meaning is confirmed.
3. Hourly weather observations are matched to the corresponding
   operational hour.
4. Relationships found in observational data indicate association,
   not necessarily causation.
"""

print(DATA_ASSUMPTIONS)
```

That last point is especially important. This is not a controlled experiment, so avoid claiming causality too quickly.

Part 6: Create useful operational features

Cell 13: Create time and system features

</> Python



▶ Run

```
analysis_df["date"] = analysis_df["timestamp"].dt.date
analysis_df["hour"] = analysis_df["timestamp"].dt.hour
analysis_df["day_of_week"] = analysis_df["timestamp"].dt.day_name()
analysis_df["is_weekend"] = (
    analysis_df["timestamp"].dt.dayofweek >= 5
).astype(int)

analysis_df["temperature_delta_f"] = (
    analysis_df["system_return_temp_f"]
    - analysis_df["system_supply_temp_f"]
)

analysis_df["pressure_gap_psi"] = (
    analysis_df["system_output_pressure_psi"]
    - analysis_df["customer_dp_psi"]
)

analysis_df["dp_deviation_from_12"] = (
    analysis_df["customer_dp_psi"] - 12
)
```

```
analysis_df["absolute_dp_deviation"] = (  
    analysis_df["dp_deviation_from_12"].abs()  
)
```

Temporary DP classification

Until the hiring manager confirms the acceptable range, do **not** claim that 10–14 psi is the official band. Use it only as a clearly labelled analytical assumption.

<> Python



▶ Run

```
ASSUMED_DP_LOWER = 10  
ASSUMED_DP_UPPER = 14  
  
analysis_df["dp_status_assumption"] = np.select(  
    [  
        analysis_df["customer_dp_psi"] < ASSUMED_DP_LOWER,  
        analysis_df["customer_dp_psi"] > ASSUMED_DP_UPPER,  
    ],  
    [  
        "Below assumed range",  
        "Above assumed range",  
    ],  
    default="Within assumed range",  
)  
  
analysis_df["within_2psi_of_target"] = (  
    analysis_df["customer_dp_psi"].between(10, 14)  
) .astype(int)
```

Label this prominently as:

Scenario only—pending confirmation of the acceptable DP range.

Cell 14: Derive plant operating-state indicators

These are analytical features, not confirmed engineering definitions.

<> Python



▶ Run

```
analysis_df["plant_1_active_assumption"] = (  
    analysis_df["plant_1_instantaneous_tons"] > 0  
) .astype(int)  
  
analysis_df["plant_2_positive_output_assumption"] = (  
    analysis_df["plant_2_instantaneous_tons"] > 0  
) .astype(int)
```

```

analysis_df["plant_operating_mode_assumption"] = np.select(
    [
        (
            analysis_df["plant_1_active_assumption"].eq(0)
            & analysis_df["plant_2_positive_output_assumption"].eq(0)
        ),
        (
            analysis_df["plant_1_active_assumption"].eq(1)
            & analysis_df["plant_2_positive_output_assumption"].eq(0)
        ),
        (
            analysis_df["plant_1_active_assumption"].eq(0)
            & analysis_df["plant_2_positive_output_assumption"].eq(1)
        ),
    ],
    [
        "Neither positive",
        "Plant 1 positive only",
        "Plant 2 positive only",
    ],
    default="Both positive",
)

```

Do not call this definitive “on/off status” until clarified.

Part 7: Exploratory analysis

Cell 15: Plot DP over time

<> Python



▶ Run

```

fig, ax = plt.subplots(figsize=(14, 5))

ax.plot(
    analysis_df["timestamp"],
    analysis_df["customer_dp_psi"],
    linewidth=0.8
)

ax.axhline(
    12,
    linestyle="--",
    linewidth=1.5,
    label="Preferred DP: 12 psi"
)

ax.set_title("Customer Differential Pressure Over Time")
ax.set_xlabel("Timestamp")

```

```
ax.set_ylabel("Customer DP (psi)")
ax.legend()
ax.grid(alpha=0.25)

plt.tight_layout()
plt.show()
```

Look for:

- long periods above or below 12 psi;
- sudden step changes;
- repeated daily patterns;
- unstable periods;
- possible plant switching events.

Cell 16: Examine the DP distribution

</> Python



▶ Run

```
fig, ax = plt.subplots(figsize=(9, 5))

ax.hist(
    analysis_df["customer_dp_psi"],
    bins=40,
    edgecolor="black",
    alpha=0.75
)

ax.axvline(
    12,
    linestyle="--",
    linewidth=1.5,
    label="Preferred DP: 12 psi"
)

ax.set_title("Distribution of Customer Differential Pressure")
ax.set_xlabel("Customer DP (psi)")
ax.set_ylabel("Number of 5-minute observations")
ax.legend()

plt.tight_layout()
plt.show()
```

Calculate basic target performance:

</> Python



▶ Run

```
dp_summary = pd.Series({
    "observations": len(analysis_df),
```

```

"mean_dp": analysis_df["customer_dp_psi"].mean(),
"median_dp": analysis_df["customer_dp_psi"].median(),
"minimum_dp": analysis_df["customer_dp_psi"].min(),
"maximum_dp": analysis_df["customer_dp_psi"].max(),
"standard_deviation": analysis_df["customer_dp_psi"].std(),
"mean_absolute_deviation_from_12":
    analysis_df["absolute_dp_deviation"].mean(),
"pct_within_assumed_10_to_14":
    analysis_df["within_2psi_of_target"].mean() * 100,
})

display(dp_summary.to_frame("value"))

```

Part 8: Correlation analysis

Cell 17: Calculate Pearson and Spearman correlations

Use both:

- **Pearson** for linear relationships;
- **Spearman** for monotonic relationships that may not be linear.

Python



Run

```

target = "customer_dp_psi"

candidate_drivers = [
    "system_output_pressure_psi",
    "system_return_temp_f",
    "system_supply_temp_f",
    "total_system_cooling_tons",
    "total_system_flow_gpm",
    "plant_1_instantaneous_tons",
    "plant_2_instantaneous_tons",
    "humidex_c",
    "temperature_delta_f",
]

correlation_results = []

for variable in candidate_drivers:
    pair = analysis_df[[target, variable]].dropna()

    pearson_r, pearson_p = stats.pearsonr(
        pair[target],
        pair[variable]
    )

    spearman_rho, spearman_p = stats.spearmanr(

```

```

        pair[target],
        pair[variable]
    )

    correlation_results.append({
        "variable": variable,
        "pearson_r": pearson_r,
        "pearson_p_value": pearson_p,
        "spearman_rho": spearman_rho,
        "spearman_p_value": spearman_p,
        "absolute_pearson_r": abs(pearson_r),
    })

correlation_table = (
    pd.DataFrame(correlation_results)
    .sort_values("absolute_pearson_r", ascending=False)
)

display(correlation_table)

```

Do not rely only on p-values. With more than 4,000 observations, even weak relationships may be statistically significant.

Focus on:

- direction;
- magnitude;
- operational relevance;
- stability across time;
- whether other variables explain the same relationship.

Cell 18: Correlation chart

</> Python



▶ Run

```

plot_data = correlation_table.sort_values("pearson_r")

fig, ax = plt.subplots(figsize=(10, 6))

ax.barh(
    plot_data["variable"],
    plot_data["pearson_r"]
)

ax.axvline(0, linewidth=1)
ax.set_title("Linear Correlation with Customer DP")
ax.set_xlabel("Pearson correlation coefficient")
ax.set_ylabel("Potential driver")

```

```
plt.tight_layout()
plt.show()
```

Part 9: Check for multicollinearity

System variables may move together. For example:

- total cooling tons;
- flow;
- plant tons;
- return and supply temperatures.

A correlation may appear important only because it is tracking another variable.

Cell 19: Correlation matrix

Python



Run

```
model_features = [
    "system_output_pressure_psi",
    "system_return_temp_f",
    "system_supply_temp_f",
    "total_system_cooling_tons",
    "total_system_flow_gpm",
    "plant_1_instantaneous_tons",
    "plant_2_instantaneous_tons",
    "humidex_c",
]

corr_matrix = analysis_df[
    [target] + model_features
].corr(method="pearson")

fig, ax = plt.subplots(figsize=(11, 9))

image = ax.imshow(
    corr_matrix,
    aspect="auto",
    vmin=-1,
    vmax=1
)

ax.set_xticks(range(len(corr_matrix.columns)))
ax.set_xticklabels(
    corr_matrix.columns,
    rotation=90
)
```

```
ax.set_yticks(range(len(corr_matrix.index)))
ax.set_yticklabels(corr_matrix.index)

fig.colorbar(image, ax=ax, label="Correlation")

ax.set_title("Correlation Matrix")

plt.tight_layout()
plt.show()
```

Cell 20: Calculate VIF

Python



Run

```
vif_data = analysis_df[model_features].dropna().copy()

vif_scaled = StandardScaler().fit_transform(vif_data)

vif_table = pd.DataFrame({
    "variable": model_features,
    "vif": [
        variance_inflation_factor(vif_scaled, i)
        for i in range(vif_scaled.shape[1])
    ]
}).sort_values("vif", ascending=False)

display(vif_table)
```

General interpretation:

- VIF below 5: usually manageable;
- VIF between 5 and 10: investigate;
- VIF above 10: strong overlap with other predictors.

Do not mechanically delete a variable only because VIF is high. Consider the operational meaning too.

Part 10: Multiple regression

This answers:

- Which variables matter after controlling for the others?
- Is the relationship positive or negative?
- How large is the estimated effect?
- Is the relationship statistically significant?

Cell 21: Fit an interpretable regression model

</> Python



▶ Run

```
regression_columns = model_features + [target]

regression_df = analysis_df[
    regression_columns
].dropna().copy()

X = regression_df[model_features]
y = regression_df[target]

X_with_constant = sm.add_constant(X)

ols_model = sm.OLS(
    y,
    X_with_constant
).fit(
    cov_type="HAC",
    cov_kws={"maxlags": 12}
)

print(ols_model.summary())
```

Using HAC robust standard errors helps account for some time-series autocorrelation and changing variance.

However, raw coefficients use different units and are hard to rank. Therefore, calculate standardized coefficients.

Cell 22: Standardized regression effects

</> Python



▶ Run

```
X_standardized = pd.DataFrame(
    StandardScaler().fit_transform(X),
    columns=X.columns,
    index=X.index,
)

y_standardized = pd.Series(
    StandardScaler().fit_transform(
        y.to_numpy().reshape(-1, 1)
    ).ravel(),
    index=y.index,
    name=target,
)
```

```

standardized_model = sm.OLS(
    y_standardized,
    sm.add_constant(X_standardized)
).fit(
    cov_type="HAC",
    cov_kwds={"maxlags": 12}
)

standardized_results = pd.DataFrame({
    "variable": model_features,
    "standardized_effect": standardized_model.params[
        model_features
    ],
    "p_value": standardized_model.pvalues[
        model_features
    ],
    "lower_95_ci": standardized_model.conf_int().loc[
        model_features, 0
    ],
    "upper_95_ci": standardized_model.conf_int().loc[
        model_features, 1
    ],
})

standardized_results["direction"] = np.where(
    standardized_results["standardized_effect"] > 0,
    "Positive",
    "Negative"
)

standardized_results["absolute_effect"] = (
    standardized_results["standardized_effect"].abs()
)

standardized_results["statistically_significant_5pct"] = (
    standardized_results["p_value"] < 0.05
)

standardized_results = standardized_results.sort_values(
    "absolute_effect",
    ascending=False
)

display(standardized_results)

```

Interpretation example:

“After controlling for the other variables, a one-standard-deviation increase in flow was associated with a 0.35-standard-deviation increase in DP.”

Do not say that until you see your actual result.

Part 11: Time-based model validation

Do not randomly split time-series data. That would allow future observations to influence the past.

Cell 23: Prepare time-based modelling data

</> Python



▶ Run

```
model_df = (
    analysis_df[
        ["timestamp", target] + model_features
    ]
    .dropna()
    .sort_values("timestamp")
    .reset_index(drop=True)
)

X_model = model_df[model_features]
y_model = model_df[target]

time_series_cv = TimeSeriesSplit(n_splits=5)

scoring = {
    "mae": "neg_mean_absolute_error",
    "rmse": "neg_root_mean_squared_error",
    "r2": "r2",
}
```

Cell 24: Compare linear and nonlinear models

</> Python



▶ Run

```
models = {
    "Linear regression": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", LinearRegression()),
    ]),

    "Ridge regression": Pipeline([
        ("imputer", SimpleImputer(strategy="median")),
        ("scaler", StandardScaler()),
        ("model", Ridge(alpha=1.0)),
    ]),
}
```

```

"Random forest": Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", RandomForestRegressor(
        n_estimators=500,
        min_samples_leaf=10,
        max_features=0.8,
        random_state=42,
        n_jobs=-1
    )),
]),

"Gradient boosting": Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("model", HistGradientBoostingRegressor(
        max_iter=300,
        learning_rate=0.05,
        max_leaf_nodes=15,
        min_samples_leaf=20,
        random_state=42
    )),
]),
}

model_comparison = []

for model_name, model in models.items():
    scores = cross_validate(
        model,
        X_model,
        y_model,
        cv=time_series_cv,
        scoring=scoring,
        n_jobs=-1,
        return_train_score=False
    )

    model_comparison.append({
        "model": model_name,
        "mean_mae": -scores["test_mae"].mean(),
        "mean_rmse": -scores["test_rmse"].mean(),
        "mean_r2": scores["test_r2"].mean(),
        "r2_std": scores["test_r2"].std(),
    })

model_comparison = pd.DataFrame(
    model_comparison
).sort_values("mean_mae")

display(model_comparison)

```

How to interpret:

- If nonlinear models perform materially better than regression, nonlinear relationships probably exist.
- If regression performs almost as well, the relationships may be mostly linear and easier to explain.
- Do not select a complex model only because its training score is higher. Use time-based validation results.

Part 12: Nonlinear relationships and feature importance

Cell 25: Train the best nonlinear model

Python



Run

```
final_model = models["Gradient boosting"]

final_model.fit(
    X_model,
    y_model
)

predictions = final_model.predict(X_model)

print("In-sample MAE:", mean_absolute_error(y_model, predictions))
print("In-sample RMSE:", mean_squared_error(
    y_model,
    predictions
) ** 0.5)
print("In-sample R²:", r2_score(y_model, predictions))
```

Use the cross-validation metrics—not the in-sample metrics—as the primary performance result.

Cell 26: Permutation feature importance

Permutation importance asks how much model performance deteriorates when each variable is scrambled.

Python



Run

```
permutation = permutation_importance(
    final_model,
    X_model,
    y_model,
    scoring="neg_mean_absolute_error",
```

```

n_repeats=20,
random_state=42,
n_jobs=-1
)

importance_table = pd.DataFrame({
    "variable": model_features,
    "importance_mean": permutation.importances_mean,
    "importance_std": permutation.importances_std,
}).sort_values("importance_mean", ascending=False)

display(importance_table)

```

Plot it:

</> Python



▶ Run

```

plot_importance = importance_table.sort_values(
    "importance_mean"
)

fig, ax = plt.subplots(figsize=(10, 6))

ax.barh(
    plot_importance["variable"],
    plot_importance["importance_mean"],
    xerr=plot_importance["importance_std"]
)

ax.set_title("Nonlinear Model: Permutation Feature Importance")
ax.set_xlabel("Increase in prediction error when variable is shuffled")
ax.set_ylabel("Variable")

plt.tight_layout()
plt.show()

```

Important distinction:

- regression coefficient shows direction and conditional effect;
- permutation importance shows predictive importance;
- correlation shows unadjusted association.

Your final driver table should combine all three.

Cell 27: Partial-dependence plots

These show whether a relationship is:

- positive;
- negative;

- flat;
- curved;
- threshold-based;
- saturating.

<> Python



▶ Run

```
top_features = (
    importance_table
    .head(4) ["variable"]
    .tolist()
)

for feature in top_features:
    fig, ax = plt.subplots(figsize=(8, 5))

    PartialDependenceDisplay.from_estimator(
        final_model,
        X_model,
        [feature],
        grid_resolution=50,
        ax=ax
    )

    ax.set_title(
        f"Partial Relationship Between {feature} and Customer DP"
    )

plt.tight_layout()
plt.show()
```

These plots can reveal findings such as:

- DP rises rapidly after flow exceeds a certain level;
- pressure has a strong positive relationship only within one operating range;
- humidex matters mostly at high cooling load;
- one plant's output has diminishing returns.

Again, only report what the actual graphs show.

Part 13: Test interactions

Likely operational interactions include:

- output pressure × system flow;
- cooling tons × flow;
- humidex × cooling tons;

- Plant 1 tons × Plant 2 tons;
- temperature differential × flow.

Cell 28: Explicit interaction regression

<> Python



▶ Run

```
interaction_df = analysis_df[
    [
        target,
        "system_output_pressure_psi",
        "total_system_flow_gpm",
        "total_system_cooling_tons",
        "humidex_c",
        "plant_1_instantaneous_tons",
        "plant_2_instantaneous_tons",
        "temperature_delta_f",
    ]
].dropna().copy()

interaction_df["pressure_x_flow"] = (
    interaction_df["system_output_pressure_psi"]
    * interaction_df["total_system_flow_gpm"]
)

interaction_df["humidex_x_load"] = (
    interaction_df["humidex_c"]
    * interaction_df["total_system_cooling_tons"]
)

interaction_df["plant1_x_plant2"] = (
    interaction_df["plant_1_instantaneous_tons"]
    * interaction_df["plant_2_instantaneous_tons"]
)

interaction_features = [
    "system_output_pressure_psi",
    "total_system_flow_gpm",
    "total_system_cooling_tons",
    "humidex_c",
    "plant_1_instantaneous_tons",
    "plant_2_instantaneous_tons",
    "temperature_delta_f",
    "pressure_x_flow",
    "humidex_x_load",
    "plant1_x_plant2",
]

X_interaction = interaction_df[interaction_features]
y_interaction = interaction_df[target]
```

```

X_interaction_standardized = pd.DataFrame(
    StandardScaler().fit_transform(X_interaction),
    columns=interaction_features,
    index=X_interaction.index,
)

interaction_model = sm.OLS(
    StandardScaler().fit_transform(
        y_interaction.to_numpy().reshape(-1, 1)
    ).ravel(),
    sm.add_constant(X_interaction_standardized)
).fit(
    cov_type="HAC",
    cov_kws={"maxlags": 12}
)

interaction_results = pd.DataFrame({
    "variable": interaction_model.params.index,
    "coefficient": interaction_model.params.values,
    "p_value": interaction_model.pvalues.values,
}).sort_values(
    "p_value"
)

display(interaction_results)

```

Be careful: interaction terms can be unstable when the original variables are strongly correlated. Treat them as supporting evidence, not absolute proof.

Part 14: Check lagged relationships

Current DP may respond to operating changes several minutes later. This is a strong senior-level addition.

Cell 29: Create lagged driver variables

<> Python



▶ Run

```

lag_source_variables = [
    "system_output_pressure_psi",
    "total_system_flow_gpm",
    "total_system_cooling_tons",
    "plant_1_instantaneous_tons",
    "plant_2_instantaneous_tons",
]

lagged_df = analysis_df.sort_values("timestamp").copy()

for variable in lag_source_variables:

```

```
for lag_intervals in [1, 2, 3, 6, 12]:
    minutes = lag_intervals * 5

    lagged_df[f"{variable}_lag_{minutes}min"] = (
        lagged_df[variable].shift(lag_intervals)
    )
```

Calculate correlation with current DP:

<> Python



▶ Run

```
lag_columns = [
    col for col in lagged_df.columns
    if "_lag_" in col
]

lag_correlations = (
    lagged_df[
        [target] + lag_columns
    ]
    .corr()[target]
    .drop(target)
    .sort_values(
        key=lambda series: series.abs(),
        ascending=False
    )
)

display(lag_correlations.head(20).to_frame("correlation_with_dp"))
```

This may reveal that a variable has more impact 10–30 minutes later than immediately.

Do not include every lag in the final dashboard. Use it to support one useful operational insight.

Part 15: Create the final driver summary

Cell 30: Combine findings

<> Python



▶ Run

```
driver_summary = (
    correlation_table[
        [
            "variable",
            "pearson_r",
            "pearson_p_value",
            "spearman_rho",
```

```

    ]
  ]
  .merge(
    standardized_results[
      [
        "variable",
        "standardized_effect",
        "direction",
        "p_value",
        "lower_95_ci",
        "upper_95_ci",
      ]
    ],
    on="variable",
    how="left",
    suffixes=("_correlation", "_regression")
  )
  .merge(
    importance_table,
    on="variable",
    how="left"
  )
)

driver_summary["regression_significant"] = (
  driver_summary["p_value"] < 0.05
)

driver_summary = driver_summary.sort_values(
  "importance_mean",
  ascending=False
)

display(driver_summary)

```

Your final driver interpretation should answer:

Question	Best analytical output
Which variable has the strongest impact?	Permutation importance and standardized regression
Positive or negative?	Standardized regression and partial-dependence plot
How significant?	Effect size, confidence interval and p-value
Nonlinear?	Model comparison and partial dependence
Interactions?	Interaction tests and segmented analysis

Question

Best analytical output

Trends?

Time series, hourly pattern and lag analysis

Part 16: Export files for Power BI

Cell 31: Select the clean Power BI table

<> Python



▶ Run

```
powerbi_columns = [
    "timestamp",
    "date",
    "hour",
    "day_of_week",
    "is_weekend",
    "customer_dp_psi",
    "system_output_pressure_psi",
    "system_return_temp_f",
    "system_supply_temp_f",
    "temperature_delta_f",
    "total_system_cooling_tons",
    "total_system_flow_gpm",
    "plant_1_instantaneous_tons",
    "plant_2_instantaneous_tons",
    "humidex_c",
    "pressure_gap_psi",
    "dp_deviation_from_12",
    "absolute_dp_deviation",
    "within_2psi_of_target",
    "dp_status_assumption",
    "plant_1_active_assumption",
    "plant_2_positive_output_assumption",
    "plant_operating_mode_assumption",
]

powerbi_df = analysis_df[powerbi_columns].copy()

display(powerbi_df.head())
```

Cell 32: Export CSV and Excel outputs

<> Python



▶ Run

```
OUTPUT_FOLDER = Path("outputs")
OUTPUT_FOLDER.mkdir(exist_ok=True)

powerbi_df.to_csv(
    OUTPUT_FOLDER / "customer_dp_powerbi_data.csv",
    index=False
)

driver_summary.to_csv(
    OUTPUT_FOLDER / "driver_summary.csv",
    index=False
)

quality_report.to_csv(
    OUTPUT_FOLDER / "data_quality_summary.csv"
)

model_comparison.to_csv(
    OUTPUT_FOLDER / "model_comparison.csv",
    index=False
)

with pd.ExcelWriter(
    OUTPUT_FOLDER / "customer_dp_analysis_outputs.xlsx",
    engine="openpyxl"
) as writer:
    powerbi_df.to_excel(
        writer,
        sheet_name="PowerBI_Data",
        index=False
    )

    driver_summary.to_excel(
        writer,
        sheet_name="Driver_Summary",
        index=False
    )

    quality_report.to_excel(
        writer,
        sheet_name="Data_Quality"
    )

    model_comparison.to_excel(
        writer,
        sheet_name="Model_Comparison",
        index=False
    )

print("Files created:")
```

Use `customer_dp_powerbi_data.csv` as your main Power BI source.

What you can do before the hiring manager replies

You can safely complete:

- workbook restructuring;
- timestamp validation;
- weather matching;
- missing-value and duplicate checks;
- descriptive statistics;
- DP time trend and distribution;
- correlation analysis;
- preliminary regression;
- nonlinear model comparison;
- draft dashboard layout.

Keep these items provisional:

- interpretation of Plant 1 and Plant 2;
- classification of plant on/off states;
- whether negative tonnage is invalid;
- whether 12 psi is a setpoint, minimum or general preference;
- any “within acceptable range” KPI;
- final operational recommendations.

How to describe your current assumptions

Place this near the beginning of your notebook and in the presentation appendix:

Preliminary assumptions pending confirmation

Hourly weather observations were matched to all 5-minute operating readings within the corresponding hour. The 12 psi value was treated as a preferred reference point, not a confirmed compliance limit. Zero and negative plant-tonnage readings were retained during the initial analysis because their operational meaning had not yet been confirmed.

That makes your work defensible.

Power BI structure after Python

Your dashboard should eventually have three pages:

Page 1 — DP Performance Overview

Show:

- average DP;
- median DP;
- minimum and maximum DP;
- average absolute deviation from 12 psi;
- percentage within the confirmed acceptable band;
- DP time series with target line;
- DP distribution;
- filters for date, hour and operating mode.

Page 2 — Driver Analysis

Show:

- ranked driver importance;
- positive/negative impact;
- scatter plots for the top three drivers;
- regression effect and confidence interval;
- selected nonlinear relationship;
- driver interpretation text.

Page 3 — Operational Conditions

Show:

- DP by plant operating mode;
- DP by hour of day;
- DP versus flow and cooling load;
- high-deviation periods;
- recommendations and operating ranges.

The strongest presentation will not claim that the statistical model has discovered causation. It will say:

“The analysis identifies conditions associated with changes in customer DP and provides testable operational hypotheses for engineering validation.”

That one sentence shows mature analytical judgment.